

Omni Suite — Studio & Code Reference

Oct 9, 2026 · @Raghav Raman

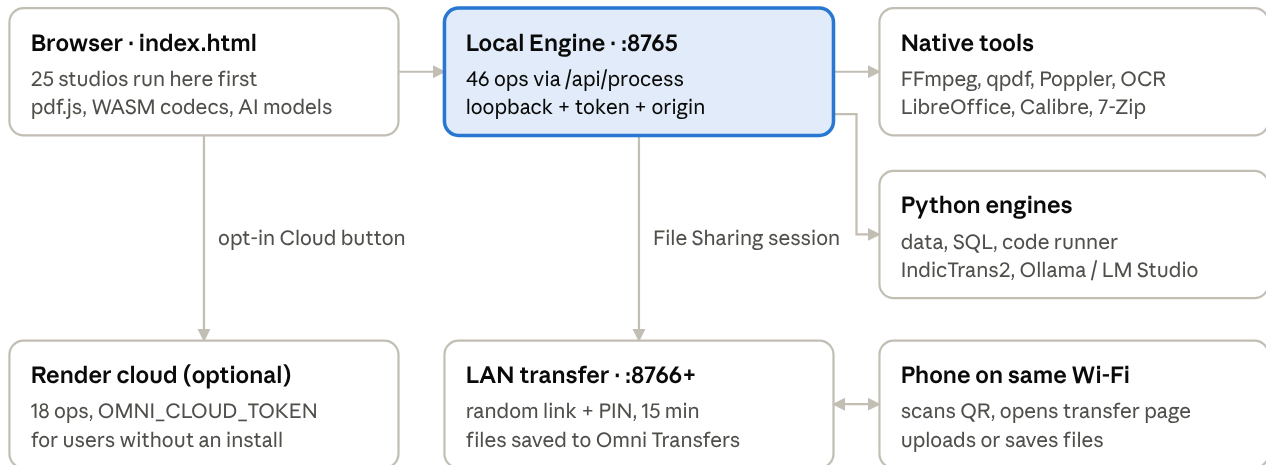
Overview and architecture

Omni Suite (repo `theraghavraman/omni_suite_light`, product name OmniConverter Studio) is one web page, `index.html`, holding 25 studios that process files on your own device. The same page runs in three modes: in the browser alone, paired with a Python Local Engine on your computer, or paired with an optional Render cloud engine.

The three execution layers

Layer	Where it runs	What it can do	Main code
Browser mode	GitHub Pages or any static host; JavaScript and WASM in the tab	Images, browser OCR, PDF rendering and light PDF edits, ZIP/GZIP, EPUB, data formats, AI models in the browser	<code>index.html</code> , <code>omni-core.js</code> , <code>omni_browser_*.js</code> , studio JS files
Local Engine	<code>omni_local_server.py</code> on <code>127.0.0.1:8765</code>	Calls native tools: FFmpeg, qpdf, Poppler, LibreOffice, Calibre, ImageMagick, Tesseract, Pandoc, 7-Zip; data/SQL engines; code runner; LAN file transfer on port 8766+	<code>omni_local_server.py</code> plus the <code>omni_*.py</code> engines
Cloud engine (optional)	A Render web service built from <code>omni-cloud-engine/ Dockerfile</code>	The same operations for people without a local install, guarded by <code>OMNI_CLOUD_TOKEN</code>	<code>omni-cloud-engine/</code> , <code>omni_render_cloud.js</code>

Studios run in the browser first and hand heavy jobs to the Local Engine



architecture · browser, Local Engine, native tools, LAN transfer, cloud

The Local Engine is the hub: the page calls it over loopback, it runs native tools and Python engines, and it opens the LAN transfer server only during a File Sharing session.

How a studio is built

Each studio is a pair: a `<button class="nav-btn" data-tab="tabX">` in the left `.nav-bar` and a `<div class="module-panel" id="tabX">` in `.container`. A click handler in an inline script toggles the `active` class on both, sets the studio's accent colours as CSS variables (`--active-studio-accent`, `--active-studio-accent-2`, `--active-studio-soft`) and scrolls the panel into view. 24 studios are written directly into `index.html`; File Sharing is injected at runtime by `file_sharing_studio.js`, which creates its own nav button and panel.

How work is routed

Every studio tries browser processing first where it can. When a job needs a native tool, the page calls the Local Engine: it reads `window.OMNI_TOKEN` (injected into the page when the engine serves it) or fetches `/api/health`, then POSTs to `/api/upload` and `/api/process` with the `X-Omni-Token` header. Outputs come back from `/api/download/<id>`. If no engine is running, browser-capable tools still work and the rest show a "Local Engine required" message.

Script load order (from `index.html`, top to bottom)

1. CDN libraries: pdf.js 3.11.174, JSZip 3.10.1, pdf-lib 1.17.1, Tesseract.js 5.1.1; SheetJS and js-yaml load from `vendor/` first, CDN as fallback.

2. Language and browser AI: `omni_language_engine.js`, `omni_language_browser.js`, `omni_language_studio.js`, `omni_browser_ai.js`, `omni_deepseek_code_ai.js`, `omni_browser_ai_ui.js`.
3. Core: `omni-core.js`, then the data modules `omni_universal_data_formats.js`, `omni_sleep_hypnogram.js`.
4. Studios and helpers: `social_media_studio.js`, `universal_data_studio.js`, `rag_studio.js`, `omni_local_ai.js`, `omni_assistant.js`, `bi_code_studio.js`, `omni_browser_studio_plus.js`, `omni_browser_docs.js`, `omni_vendor_loader.js`, `office_tools_studio.js`, `office_find_replace.js`, `diagram_forge_studio.js`.
5. Browser engines: `omni_browser_epub.js`, `omni_browser_ocr.js`, `omni_browser_images.js`, `photo_editor.js`, `omni_browser_audio.js`, `omni_browser_video.js`, `omni_browser_archive.js`.
6. Last: `omni_render_cloud.js` and `file_sharing_studio.js`.

The page also carries 24 inline scripts (about 180 KB) for tab switching, upload pickers, the command palette, diagnostics, Batch Lab and data routing.

Security model in one paragraph

The Local Engine binds to loopback only and refuses requests whose TCP peer is not loopback. Processing requests need the per-launch `X-Omni-Token` and an allowed `Origin` (`http://127.0.0.1:8765`, `http://localhost:8765`, `https://theraghavraman.github.io`, plus `OMNI_ALLOWED_ORIGINS`). It runs a fixed list of operations, never arbitrary shell commands, and cleans temporary files. The one exception to loopback is the temporary LAN transfer server, which only serves a single random session link for up to 15 minutes.

Getting started

Double-click `Omni.command` (macOS/Linux) or `Omni.bat` (Windows); it installs whatever is missing on first run and then opens `http://127.0.0.1:8765/`. Browser-only use needs nothing: open the GitHub Pages site.

What the one-click launch does

1. `Omni.command` / `Omni.bat` hands off to `start_omni.command` / `start_omni.bat`.
2. The start script prefers `.venv/bin/python` (or `.venv\Scripts\python.exe`), else system Python 3, and runs `omni_bootstrap.py`. If Python is missing it runs the platform installer first.

3. `omni_bootstrap.py` → `verify()` checks that `certifi`, `omni_local_server`, `omni_data_engine` and `omni_platform` import, and that the 10 offline assets exist in `vendor/` (`pdf.js` + `worker`, `JSZip`, `pdf-lib`, `Tesseract.js` + `worker` + core WASM + `engine` language, `SheetJS`, `js-yaml`).
4. If verification fails, `run_setup()` runs `install_windows.ps1` (PowerShell) or `install_system_tools.command` (bash), then verifies again.
5. `health()` probes port 8765. An engine with `engine_api_version` ≥ 4 is reused and the browser opens. An older engine, or any `omni_local_server.py` process holding the port, is stopped by `stop_stale_engine()`; a foreign process on 8765 aborts the launch with its PID.
6. Finally it runs `omni_local_server.py` in the `venv`.

Setup scripts

Script	Platform	Job
<code>install_system_tools.command</code>	macOS, Linux	Detects Homebrew, apt, dnf, pacman or zypper; installs FFmpeg, qpdf, Poppler, ImageMagick, Ghostscript, Tesseract + languages, LibreOffice, Calibre, Pandoc, 7-Zip; builds <code>.venv</code> from <code>requirements-local.txt</code> ; runs <code>prepare_offline.py</code>
<code>install_windows.ps1</code>	Windows	Same, through winget, including Python itself
<code>setup_and_start.command</code> / <code>.bat</code>	All	Runs the installer, then the launcher
<code>prepare_offline.py</code>	All	<code>download(rel, url)</code> fetches pinned browser assets into <code>vendor/</code> so the local page works with no internet
<code>prepare_language_models.py</code>	All	Materialises IndicTrans2 / IndicXlit model files under <code>language-models/</code>
<code>verify_local_environment.py</code>	All	Strict check: <code>check_python()</code> , <code>check_native()</code> , <code>check_offline_assets()</code> , <code>pip_check()</code> , <code>check_python_runtime()</code>

Manual start

```
python3 prepare_offline.py      # once, while online
python3 omni_local_server.py    # then open http://127.0.0.1:8765/
```

Offline behaviour. When the engine serves `index.html` it rewrites CDN script URLs to the matching `vendor/` files if they exist, sets `OMNI_PDF_WORKER` and `OMNI_TESSERACT_OPTIONS` to local paths, and injects `window.OMNI_TOKEN`. After `prepare_offline.py` has run once, the local page needs no internet. Browser AI models and the QR designer's styling library are still fetched on first use.

Environment variables

Variable	Default	Effect
<code>OMNI_HOST</code> / <code>OMNI_PORT</code>	<code>127.0.0.1</code> / <code>8765</code>	Engine bind address and port
<code>OMNI_TOKEN</code>	random per launch	Fixes the engine token
<code>OMNI_ALLOWED_ORIGINS</code>	none	Extra browser origins allowed to call the engine (comma-separated)
<code>OMNI_MAX_UPLOAD</code>	8 GB	Engine upload limit
<code>OMNI_TRANSFER_PORT</code> / <code>OMNI_TRANSFER_TTL</code> / <code>OMNI_TRANSFER_MAX_UPLOAD</code>	<code>8766</code> / <code>900 s</code> / 8 GB	LAN file transfer server
<code>OMNI_TRANSFER_DIR</code>	<code>~/Downloads/Omni Transfers</code>	Where files from a phone are saved
<code>OMNI_AI_PROVIDER</code>	<code>auto</code>	Local LLM: <code>ollama</code> , <code>lmstudio</code> or auto-detect
<code>OMNI_OLLAMA_URL</code> / <code>OMNI_LMSTUDIO_URL</code>	<code>:11434</code> / <code>:1234</code> on loopback	Local LLM servers
<code>OMNI_AI_MODEL</code> / <code>OMNI_AI_EMBED_MODEL</code> / <code>OMNI_AI_TIMEOUT</code>	<code>auto</code> / <code>none</code> / 180–300 s	Model choice and timeout
<code>OMNI_ALLOW_LOCAL_DB</code> / <code>OMNI_ALLOWED_DB_HOSTS</code>	<code>1</code> / any	Database Studio connection limits
<code>OMNI_LANGUAGE_MODEL_ROOT</code> / <code>OMNI_INDICXLIT_MODEL_ROOT</code>	<code>language-models/</code>	Language Engine model folders

Studio catalogue

The 25 studios fall into six groups. Each entry gives the panel id, the code that drives it, its main functions and the Local Engine operations (`op` values sent to `/api/process`) it uses.

Document studios

A shared pattern runs through all four: a browser engine registers a capture-phase click handler with `intercept(buttonId, {plan, run})`. If `plan()` says the browser can do the job, it runs and stops the click; otherwise the click falls through to the inline Local Engine handler.

PDF Suite — `tabPdf`

Turns PDFs into images and merges or splits PDFs, entirely in the browser.

Feature	Controls	How it works
PDF → images (JPG, PNG, WebP, AVIF)	<code>pdf2imgInput</code> , <code>pdf2imgFormat</code> , <code>pdf2imgDpi</code> (150/200/300), <code>pdf2imgPages</code> , <code>pdf2imgBtn</code>	pdf.js renders each page to a canvas; <code>canvas.toBlob</code> encodes it. AVIF is disabled where the browser cannot encode it
Download all as ZIP	<code>pdf2imgDownloadAll</code>	JSZip packs the rendered pages
Export pages to a folder	<code>pdf2imgFolderExport</code>	File System Access API <code>showDirectoryPicker</code> (Chromium only)
Merge PDFs	<code>pdfMergeInput</code> , <code>pdfMergeBtn</code>	pdf-lib <code>PDFDocument.create()</code> + <code>copyPages</code>
Extract pages	<code>pdfSplitInput</code> , <code>pdfSplitRange</code> , <code>pdfSplitBtn</code>	pdf-lib; ranges parsed by <code>parsePageRange("1-3,5", total)</code>

Code: `omni-core.js` (`loadPdfForImg`, `openPdfForImg`, `parsePageRange`, `resetPdf2Img`). Native equivalents for large files live in the Local Engine studio: `pdf_merge`, `pdf_split`, `pdf_compress` (qpdf), `pdf_render` (Poppler `pdftoppm`), `pdf_to_text` (`pdftotext`).

Office Studio — `tabWord`

Converts Word, text, PDF, spreadsheets and presentations, and edits text inside Office files without converting them. Seven cards:

Card	Controls	Browser path (omni_browser_docs.js)	Engine path
1. DOCX → PDF / TXT	w2pInput , w2pRunBtn , w2pTxtBtn	readDocx → writePdf (Unicode, embedded Noto/Roboto fonts)	—
2. PDF → Word	p2wInput , p2wMode (editable text, visual replica, browser fallback), p2wRunBtn	pdfToModel → writeDocx	pdf_to_docx (Poppler pdftotext → DOCX)
3. Text / Markdown → DOCX & PDF	t2dTitle , t2dStyle , t2dDocxBtn , t2dPdfBtn	readMarkdown / readPlain → writeDocx / writePdf	text_document (LibreOffice)
4. Office format converter	officeConvertInput , officeConvertTarget (PDF, DOCX, DOC, DOCM, DOT/X/M, ODT, OTT, FODT, XLS/X/M/B, XLT/X...), quick buttons PPT→PDF, PDF→PPTX, CSV→XLSX, Excel/ODS→XLSX, PPT/ODP→PPTX	officePlan decides; officeRoute → readDocument + writeDocument, or convertSheet (all sheets kept)	office_convert (LibreOffice soffice); pdf_to_ppt (pdftoppm pages → PPTX)
5. Presentation converter	officePresentationInput , officePresentationTarget (PDF, PPT, PPTX, PPTM, PPS/X, POT/X/M, ODP, OTP, FODP)	readPptx → outline PDF or text formats	office_convert for pixel-exact slides
6. Spreadsheet converter	officeSpreadsheetInput , officeSpreadsheetTarget (XLS...FODS, CSV, TSV)	convertSheet via SheetJS	office_convert
7. Find & Replace in Office files	ofrInput , rules (ofrAddRule , ofrImport / ofrExport CSV), options (case, whole word, regex, preserve case, headers, notes, charts, comments, masters, properties, ZIP output), ofrScan , ofrReplace	office_find_replace.js	—

omni_browser_docs.js (65 KB, exported as window.OMNI_DOCS) is the shared document model. Readers: readDocx , readOdt , readRtf , readHtml , readMarkdown , readPlain , readPptx , dispatched by readDocument . Writers: writePdf (with layout , para ,

`table` , font fallback via `fontFor` and raster fallback `rasterPng` for unsupported scripts), `writeDocx` , `writeOdt` , `writeHtml` , `writeMarkdown` , `writeText` , dispatched by `writeDocument` .

`office_find_replace.js` ("Redmark Forge", `window.OMNI_FIND_REPLACE`) edits the XML inside DOCX/DOCM/DOTX, PPTX/PPTM/PPSX/POTX, XLSX/XLSM/XLTX and ODT/ODS/ODP packages. `segmentsOf` joins a paragraph's runs so matches can cross run boundaries; `replaceInSegments` writes the replacement into the run where the match starts, keeping its formatting; matches crossing a tab, break or field are skipped and reported. `partsFor` picks which XML parts to touch, `processFile` rewrites only changed parts, `renderReport` lists every match.

Office Tools — `tabOfficeTools`

A browser-only Word, Excel, PowerPoint and Notepad workspace with New, Open, Save, Save As and Print. Code: `office_tools_studio.js` (126 KB, 136 functions, `window.REDMARK_OFFICE_TOOLS`); libraries load on demand through `omni_vendor_loader.js` .

Mode	What you can do	Key functions	Libraries
Word / Writer	Rich text, 14 fonts, sizes 8-72, headings, lists, links, images, tables, page breaks, find/replace, zoom; save DOCX, ODT, RTF, HTML, PDF, MD, TXT	<code>wordOpen</code> , <code>wcmd</code> , <code>insertTable</code> , <code>tableOp</code> , <code>wordFind</code> / <code>wordReplaceAll</code> , <code>editorIR</code> → <code>irToModel</code> → <code>buildDocx</code> / <code>buildRtf</code> , <code>wordExport</code>	mammoth (open), docx (save), DOMPurify

Mode	What you can do	Key functions	Libraries
Excel / Calc	Grid with formulas, number formats (₹, \$, %, dates), colours, filter, freeze, sort, insert/delete rows and columns, multiple sheets, copy/paste TSV, undo/redo	<code>renderGrid</code> , <code>ensureFormulaEngine</code> , <code>cellValue</code> , <code>shiftFormula</code> , <code>sortRange</code> , <code>xUndoRedo</code> , <code>readWithExcelJS</code> / <code>readWithSheetJS</code> , <code>sheetExport</code>	ExcelJS, SheetJS, fast-formula-parser, Formula.js
PowerPoint / Impress	Slides with 6 layouts and 6 themes, images, reorder, duplicate, full-screen present	<code>newDeck</code> , <code>renderDeck</code> , <code>deckMutate</code> , <code>buildPptx</code> , <code>present</code>	PptxGenJS
Notepad	Plain text with LF/CRLF, UTF-8 / UTF-8 BOM / UTF-16 LE, wrap, find/replace	<code>noteOpen</code> , <code>decodeBytes</code> , <code>noteExport</code>	—

Unsaved work is kept as a draft in `localStorage` (`redmark-office-draft-<mode>` , via `scheduleDraft` / `loadDraft`).

EPUB Studio — `tabEpub`

Converts between EPUB, PDF and other ebook formats.

Card	Controls	Browser path (<code>omni_browser_epub.js</code>)	Engine path
1. EPUB → PDF, JPG pages, illustrations ZIP, clean TXT	<code>epubInput</code> , <code>epubTargetAction</code> , <code>epubRunBtn</code>	<code>readEpub</code> (spine order, images) → <code>OMNI_DOCS.writePdf</code> , page rendering, image extraction	—

Card	Controls	Browser path (omni_browser_epub.js)	Engine path
2. PDF → EPUB 3	p2eInput , p2eTitle , p2eAuthor , p2eMode (illustrated, fixed-layout, reflowable)	pdfPagesToImages + writeEpub	—
3. Native ebook bridge	ebookNativeInput , ebookNativeTarget (EPUB, AZW3, MOBI, PDF, DOCX, FB2, HTMLZ, OEB, TXT, RTF, LRF, PDB, PMLZ, RB, SNB, TCR, TXZ, ZIP)	readEbook / writeEbook handle EPUB, FB2, CBZ, RTF, HTMLZ, DOCX, PDF, TXT	ebook_convert (Calibre ebook-convert) for Kindle and legacy formats

omni_browser_epub.js (26 KB, window.OMNI_EPUB) depends on OMNI_DOCS ; its readers are readEpub , readFb2 , readCbz ; writers writeEpub , writeFb2 , writeRtf .

Image and media studios

Each of these studios has a browser card powered by an omni_browser_*.js engine that loads heavy codecs only when first used, plus a "Native ... Bridge" card that sends the file to the Local Engine. The native bridge cards share one inline helper, bindDedicated , which checks /api/health for engine API ≥ 4 and the required tool before uploading.

Image Tools — tabImages

Card	Controls	Code	Engine path
1. Batch format converter (PNG, JPG, WebP, AVIF)	imgInput , imgTargetFmt , imgQualitySelect (95/85/75 %), imgRunBtn , imgDownloadAll	omni_browser_images.js : decode → encode ; per-file error isolation	—
2. Images → PDF	i2pInput , i2pBtn	pdfEncode with pdf-lib	—

Card	Controls	Code	Engine path
3. Resize, DPI/PPI, photo sizes	<code>imgResizeInput</code> , width/height in px or mm, <code>imgResizeLock</code> (crop, fit, stretch), <code>imgResizeDpi</code> (72–600), presets: Indian passport photo, 2×2 ID, signature	<code>renderResized</code> , <code>mm2px</code> , <code>setDpi</code> writes DPI into JPEG/PNG, <code>encodeUnder</code> meets a max file size	—
4. Native image bridge	<code>imageNativeInput</code> , <code>imageNativeTarget</code> (PNG, JPG, WebP, AVIF, GIF, TIFF, BMP, SVG, ICO, HEIC, JPEG XL, JPEG 2000, TGA, DDS, EXR, HDR, PDF)	Browser first where possible	<code>image</code> (ImageMagick <code>magick / convert</code> , <code>rsvg-convert</code> for SVG; params <code>format</code> , <code>width</code> , <code>height</code> , <code>quality</code> , <code>density</code> , <code>strip</code>)
5. Photo Editor	<code>peInput</code> ; tools Crop, Adjust, Filters, Blur, Draw, Text, Resize & Save	<code>photo_editor.js</code>	—

`omni_browser_images.js` (26 KB, `window.OMNI_IMAGES`) decodes EXIF orientation (`jpegOrientation`), SVG, TIFF (`utif`) and HEIC/HEIF (`heic2any`), and has its own encoders: `bmpEncode`, `tgaEncode`, `icoEncode`, `tiffEncode`, `gifEncode` (`gifenc`), `avifEncode` (@jsquash/avif WASM), `pdfEncode`, `svgEncode`.

`photo_editor.js` (40 KB, `window.OMNI_PHOTO_EDITOR`) is a single-photo editor with a 25-step history (8 steps for photos over 8 megapixels, via `histLimit`) (`pushHist`, `undo`, `redo`), hold-to-compare, crop with 8 aspect presets (`setAspect`, `cropDrag`, `applyCrop`), rotate/flip/straighten, light and colour sliders plus filter presets (`buildAdjust`, `buildFilterThumbs`, `autoEnhance`), blur and pixelate brushes or boxes and black boxes for faces or number plates (`paintBrush`, `stampEffect`, `boxBlur`), pen, highlighter, line, arrow, box and circle (`drawShape`), text in 5 styles (`drawText`), resize presets 50 %, 25 %, 1080 px, 1920 px, 4K (`applyResize`), and export to JPEG, PNG, WebP or AVIF (`exportBlob`) or the clipboard.

Image to Text (OCR) — `tab0cr`

Extracts text from images and scanned PDFs in 29 OCR languages, then translates or transliterates it.

- Controls: `ocrInput`, `ocrLanguage`, `ocrRunBtn`, copy/save buttons; translation `ocrTranslateSource` / `ocrTranslateTarget` (24 Indian and other languages); transliteration `ocrTranslitTarget`.
- Code: `omni-core.js` (`loadOcrFile`, `translateOcrText`, `transliterateOcrText`) with Tesseract.js 5.1.1; `omni_browser_ocr.js` adds multiple images and scanned PDFs (`pdfPages`), image clean-up before recognition (`prepare`), one reused Tesseract worker, a searchable-PDF download with an invisible text layer, and a transliteration fallback that works without the language model.
- Engine path: `ocr` (native Tesseract, param `lang`); translation through `language_translate` / `language_transliterate`.

Audio Studio — `tabAudio`

- Browser card: `audioInput`, start/end seconds, `audioSpeedSelect` (0.5×–2×), `audioProcessBtn`. `omni_browser_audio.js` (`window.OMNI_AUDIO0`) adds a waveform with drag-to-select (`computePeaks`, `draw`), speed change that keeps pitch (`timeStretch`, WSOLA), fades, normalise, mono/stereo, resampling (`resampleTo`), and export to WAV, MP3 (lamejs), M4A/AAC (mp4-muxer + WebCodecs), Opus/WebM (webm-muxer), AIFF and AU (`encodeWav`, `encodeMp3`, `encodeWithWebCodecs`, `encodeAiff`, `encodeAu`).
- Native bridge: `audioNativeTarget` with 18 formats (MP3, WAV, M4A, AAC, FLAC, OGG, OGA, Opus, WMA, AMR, AIFF, AC-3, E-AC-3, AU, CAF, W64, WavPack, MKA) → `media op` (FFmpeg; params `format`, `start`, `end`, `audio_only`).

Video Studio — `tabVideo`

- Browser card: `videoInput`, start/end seconds, `videoSpeedSelect`, `videoActionSelect` (trim, extract audio as WAV, capture frame as JPG). `omni_browser_video.js` (`window.OMNI_VIDEO0`) uses Mediabunny 1.58.1 (WebCodecs) to cut without re-encoding, convert MP4/WebM/MOV/MKV, resize, compress, rotate, change speed with pitch kept, extract audio to M4A/MP3/WAV (`convertVideo`, `extractAudio`), grab exact frames (`frameAt`), export frame sequences as ZIP (`framesZip`) and make GIFs (`makeGif`).

- Native bridge: `videoNativeTarget` with 23 formats (MP4, MKV, WebM, MOV, AVI, FLV, MPEG, MPG, M4V, 3GP, 3G2, MPEG-TS, M2TS, MTS, VOB, WMV, ASF, OGV, MXF, NUT, IVF, GIF, APNG) → `media` op (FFmpeg; params `scale`, `keep_video`, `video_only`).

Compressor — `tabCompress`

Card	Controls	Code	Engine path
Compress	<code>compressInput</code> , <code>compressFormat</code> (ZIP, GZIP, DEFLATE, Brotli), <code>compressLevel</code> (store, 1, 3, 6, 9)	<code>omni-core.js</code> <code>streamCompress</code> (CompressionStream API) and JSZip; <code>omni_browser_archive.js</code> adds 7z, AES-256 password ZIP, TAR, TAR.GZ/XZ/BZ2, BZIP2, XZ via 7z-wasm in a worker (<code>run7z</code>)	<code>zip</code> , <code>gzip</code>
Decompress	<code>decompressInput</code> (ZIP, 7z, RAR, GZIP, DEFLATE), <code>decompressSaveAll</code>	<code>streamDecompress</code> , <code>extractNativeArchive</code> ; 7z-wasm also opens RAR, TAR, XZ, BZIP2, CAB, ISO (<code>sniff</code> detects type)	<code>archive_extract</code> (7-Zip)
Native archive bridge	<code>archiveNativeInput</code> , <code>archiveNativeTarget</code> (ZIP, TAR, GZIP, BZIP2, XZ, 7z)	—	<code>archive</code> (7-Zip/Python <code>tarfile</code>)

Social Media — `tabSocialMedia`

Downloads public media from a pasted link. Code: `social_media_studio.js` (23 KB, `window.OMNI_SOCIAL_MEDIA_DOWNLOADER`).

- `runAnalyze` → `detect` classifies the URL. Direct media links are fetched straight away (`analyzeDirect`, `downloadDirect`).
- Otherwise `browserResolve` reads the public page (optionally through `r.jina.ai`, which sees the URL) and `findMediaUrls` scans for media; `hostedResolve` calls the optional Social Resolver service at `window.OMNI_SOCIAL_RESOLVER_URL` (`/api/resolve`).

3. If neither works and the Local Engine is up, `localJob` sends `social_download` (yt-dlp; params `url`, `mode`, `quality`, `info_only`), then downloads the result from `/api/download/<id>`.
4. `offerStudios` suggests the next studio (Video, Audio, Image) for the downloaded file.

The header states the engine does no DRM bypass, credential harvesting or login automation.

Data and code studios

`omni_browser_studio_plus.js` (37 KB, `window.OMNI_BROWSER_PLUS`) is the browser engine behind most of this group. It intercepts the Data Studio, Data Clean and Batch Lab buttons and runs them with `sql.js` (SQLite in WASM), `hyparquet` / `hyparquet-writer` (Parquet) and `SheetJS`, so they work on GitHub Pages; anything it cannot handle falls through to the Local Engine and `omni_data_engine.py`.

Data Studio — `tabData`

Card	Controls	Browser path	Engine op
1. Universal data converter	<code>dataConvertInput</code> , <code>dataConvertTarget</code> (CSV, TSV, JSON, JSONL, YAML, XML, HTML, Markdown, XLSX, ODS, HDF5, Parquet, Feather/Arrow, ORC, Avro, SQLite, DuckDB, SQL INSERT/DDL, MessagePack, JSON Schema), <code>dataConvertDialect</code> (8 SQL dialects), <code>dataConvertTable</code>	<code>readRows</code> → <code>writeRows</code> (<code>toSql</code> , <code>toXml</code> , <code>toMd</code> , <code>inferTypes</code> , <code>sqlJs</code> , <code>parquetWriter</code>)	<code>data_convert</code> (<code>read_data</code> → <code>write_data</code>)
Profile dataset	<code>dataProfileRun</code>	—	<code>data_profile</code>
2. SQL dialect translator	<code>sqlTranslateInput</code> , <code>sqlSourceDialect</code> , <code>sqlTargetDialect</code> (filled by <code>fillDialects</code>)	—	<code>sql_transpile</code> (SQLGlot, 24 dialects)

Card	Controls	Browser path	Engine op
3. Schema and migration generator	<code>schemaInput</code> , <code>schemaDialect</code> , <code>schemaTable</code>	—	<code>schema_generate</code> (<code>dataframe_sql</code>)
4. NoSQL bridge	<code>nosqlInput</code> , <code>nosqlSource</code> (JSON/Mongo Extended JSON, JSONL, BSON, DynamoDB JSON), <code>nosqlTarget</code> (+ flattened CSV/SQL, Cassandra CQL, Neo4j Cypher)	DynamoDB JSON via <code>ddbAttr</code>	<code>nosql_convert</code>
5. Unstructured text and log bridge	<code>textBridgeInput</code> , <code>textBridgeTarget</code> (JSONL, JSON, CSV)	yes	<code>text_lines</code>
6. BLOB / binary bridge	<code>blobBridgeInput</code> , <code>blobBridgeSource</code> (raw, Base64, hex), <code>blobBridgeTarget</code> (+ SQL hex literal)	<code>toBytes</code> , <code>b64</code>	<code>blob_convert</code>

Inline handlers: `runData` , `runProfile` , `runSql` , `runSchema` , `runNosql` .

Universal Data and Signal — `tabUniversalData`

Loads geospatial, weather, telemetry, scientific, audio-signal and encoded files, previews and charts them, and exports to another representation.

- Bring data in: `udsInput` (file) or `udsUrl` + `udsFetch` , with presets for an OSRM route → GeoJSON and Open-Meteo weather JSON (`fetchURL`).
- Inspect: `udsVizType` (numeric chart, map, waveform), `udsField` , `udsVisualize` (`draw` , `drawWave` , `renderGeo`).
- Convert: `udsFormat` (JSON, CSV, TSV, NDJSON, YAML, XML, HTML report, Markdown, GeoJSON, SVG, PNG, WAV sonification, hex, Base64, WebM video, plus NetCDF, HDF5, FITS, Parquet, Arrow, Avro, ORC, SQLite, SAM/BAM, PCAP, DICOM through the Local Engine) → `udsExport` (`exportAs` , `sonify` , `exportVideo` , `htmlReport`).
- Encoded bridge: `udsEncodedMode` (hex ↔ Base64 ↔ text) → `encodedConvert` .

Code: `universal_data_studio.js` (23 KB, `window.OMNI_UNIVERSAL_DATA`: `parseInput`, `parseCSV`, `parseGPX`, `parseKML`, `normalize`, `numericFields`);
`omni_universal_data_formats.js` (26 KB, `window.OMNI_UDS_FORMATS`: `convertBrowser`, `localConvert` via `data_convert` / `scientific_convert`, `parseNewick` for phylogenetic trees, `targetsFor`); `omni_sleep_hypnogram.js` (19 KB, `window.OMNI_SLEEP_HYPNOGRAM`) reads EDF/EDF+ sleep recordings (`parseEDF`, `parseTAL`, `decodeHypnogram`), computes sleep metrics and exports stage CSV, JSON or SVG.

Database Studio — `tabDatabaseStudio`

A temporary SQL sandbox that runs SQLite in the browser (`sql.js`).

- Session: `dbSandboxInit` (`sqlDb`), `dbSandboxReset`, `dbSandboxExport` (download `.sqlite`), `dbSandboxImport` (create a table from a CSV/JSON/XLSX file into `dbSandboxTable`), `dbSandboxSchema`.
- Transactions: BEGIN, COMMIT, ROLLBACK, SAVEPOINT, ROLLBACK TO, RELEASE buttons.
- Run: `dbSandboxSql` + `dbSandboxRun` (`runSql`), `dbSandboxExplain`; 10 examples in `dbSandboxExample` (DDL, CRUD, transactions, savepoints, joins, window functions, CTEs, views, triggers, built-in functions).
- The engine also has read-only `database_tables` and `database_query` ops (`omni_platform.database_query`, which blocks mutating SQL and checks `OMNI_ALLOWED_DB_HOSTS`), but no current panel calls them.

Data Clean — `tabDataClean`

`cleanInput`, `cleanTarget` (CSV, JSON, JSONL, XLSX, Parquet, Markdown).

- `cleanBtn` → `cleanRows` in the browser, or `data_clean` (`omni_platform.clean_dataframe`): removes duplicates and empty rows, trims whitespace, normalises column names to snake_case, fills numeric nulls.
- `privacyBtn` → `privacyScan` in the browser, or `privacy_scan`: flags likely emails, phones, IPs, URLs, PAN-like, Aadhaar-like (Verhoeff check, `verhoeff`) and card-like numbers (Luhn check, `luhn`), and prints a SHA-256 fingerprint. It is a review aid, not a compliance check.

BI Interchange — `tabVizInterchange`

Reads BI and visualisation definitions (Tableau `.twb` XML, JSON models and similar) and turns them into portable outputs.

- Controls: `vizInterchangeInput`, `vizAnalyzeBtn`, `vizOutputMode` (OMNI-VIZ JSON, SQL query, Excel functions, Excel workbook, conversion audit), `vizSqlDialect` (ANSI, Oracle, PostgreSQL, SQL Server, MySQL, Snowflake, BigQuery, DuckDB), `vizExcelTable`, `vizGenerateBtn`, `vizDownloadBtn`.
- Code: `bi_code_studio.js` (31 KB, `window.OMNI_BI_INTERCHANGE`): `biReadFile`, `extractTableauXml`, `extractJsonModel`, `biAnalyze`, `biSql`, `biExcel`, `biGenerate`.

Code Studio — `tabCodeStudio`

- Translator: `codeInput` / `codeFileName`, `codeSourceLanguage` (auto + 15 languages), `codeTargetLanguage` (pseudo code + 15), `codeConvertBtn`, `codePseudoBtn`. Code in `bi_code_studio.js` (`window.OMNI_CODE_STUDIO`): `detectLanguage`, `translateSyntax`, `pythonToJs`, `jsToPython`, `pseudoCode`; a confidence panel reports how much was translated.
- Execution lab: `codeLabLanguage` (13 languages), source, stdin, `codeLabTimeout`. `codeLabRunBrowser` runs JavaScript, TypeScript and Python in the page (`browserRun`); `codeLabRunLocal` sends `code_execute` to the engine (`omni_code_runner.execute`), which runs any installed toolchain on your machine — keep the engine loopback-only.

Batch Lab — `tabBatchLab`

`batchInput`, `batchOp` (data conversion or privacy scan), `batchTarget` (13 formats). `batchRunBtn` → `runBatch` / `batchOne` per file in the browser, or `batch_convert` (`omni_platform.batch_manifest`) on the engine. Each file succeeds or fails on its own; `batchRetryBtn` reruns only failures; `batchDownloadBtn` saves a JSON manifest.

AI and language studios

Three AI stacks are kept deliberately separate: browser models through Transformers.js 4.3.0 (RAG, Assistant, DeepSeek helper, browser translation), local LLMs through the engine (Ollama or LM Studio), and the Indic language models (IndicTrans2, IndicXlit) on the engine. No studio sends text to a cloud AI service.

Private RAG — `tabRAGStudio`

Builds a private knowledge base from your files in the browser and answers questions from it.

1. **Index.** `ragFiles` → `queueFiles` → `indexPendingFiles` → `extractFile` (PDF text via `pdf.js`, DOCX text via `JSZip`, anything else read as plain text) → `splitText` into 1,200-character chunks with 180 characters of overlap → stored in IndexedDB `omni-private-rag-v1` (stores `chunks`, `meta`). `ragRepo` indexes the whole `theraghavraman/omni_suite_light` repository; `ragClear` wipes the index.
2. **Embed.** `ragEmbed` → `embedTexts` runs `Xenova/all-MiniLM-L6-v2` in a Web Worker (`rag_embed_worker.js`) so the page stays responsive.
3. **Retrieve.** `search` mixes lexical scoring (`lexical`) with cosine similarity (`cosine`) and returns the top `ragTopK` chunks.
4. **Answer.** `ragUseLLM` = local small LLM (`onnx-community/gemma-3-270m-it-ONNX`) or retrieval only; `extractiveAnswer` is the fallback. Sources are listed with each answer.
5. **Save to GitHub.** `ragGithubRepo`, `ragGithubFolder`, `ragGithubBranch`, `ragGithubToken` → `saveSelectedToGitHub` commits files through the GitHub contents API; `ragGithubRemember` keeps the token, `ragGithubForget` deletes it.

Code: `rag_studio.js` (32 KB, 46 functions), `rag_embed_worker.js` (1.4 KB).

Language Engine — `tabLanguageStudio`

Translation and transliteration across 24 languages (English plus 23 Indian languages), with no third-party web fallback.

- Controls: `languageSource`, `languageTarget`, `languageTranslateBtn`; `languageTranslitTarget` (Roman or 22 scripts), `languageTranslitBtn`; `languageRefreshStatus` shows which models are installed.
- Browser path: `omni_language_browser.js`
(`window.OMNI_LANGUAGE_BROWSER_ADAPTER`) runs Hindi ↔ English with `Xenova/opus-mt-en-hi` and `Xenova/opus-mt-hi-en`; `omni_browser_studio_plus.js` adds a rule-based transliterator (`brahmicToLatin`, `brahmicToBrahmic`, `latinToDeva`, `builtInTransliterate`).
- Engine path: `omni_language_engine.js` (`window.OmniLanguageEngine`) calls `language_translate` and `language_transliterate`. `omni_language_engine.py` loads IndicTrans2 distilled models (`indictrans2-en-indic-dist-200M`, `indictrans2-indic-en-dist-200M`, `indictrans2-indic-indic-dist-320M`) with `IndicTransToolkit`,

and IndicXlit (`ai4bharat.transliteration.XlitEngine`) for Roman ↔ Indic. Models live in `language-models/`, prepared by `prepare_language_models.py` and `requirements-language.txt`.

- UI glue: `omni_language_studio.js` (`refresh`, `translate`, `transliterate`). The OCR studio reuses the same engine.

Omni Assistant (site-wide panel)

A floating help panel on every studio, not a tab. `omni_assistant.js` (26 KB) answers questions about the suite from the curated `knowledge-base/` files (`omni-suite.md`, `omniconverter-guide.md`, listed in `manifest.json`) and, when useful, the Private RAG corpus.

- Flow: `ask` → `fastAnswer` / `intentAnswer` for common questions → `retrieve` (lexical `lex` + embeddings `emb` with `Xenova/all-MiniLM-L6-v2`) → `generate` with `gemma-3-270m-it` → `cleanAssistantAnswer` + `uniqueSources`.
- `studioContext` knows the open studio; `studioAction` / `openStudio` can jump to the right one. Panel state persists through `saveUiState` / `restoreUiState`.
- `omni_local_ai.js` (`window.OmniLocalAI`) is the shared Transformers.js runtime for embeddings and chat (`ensureEmbed`, `ensureChat`, `warmChat`, `generate`).

AI Assist buttons inside studios

`omni_browser_ai_ui.js` adds one AI Assist control to each studio without touching the conversion paths. `actionDefs(panelId)` sets the actions per studio, for example: PDF — summarise, explain, key points; Office and EPUB — summarise, rewrite, themes; Images — describe, detect objects, remove background, depth map; OCR — advanced OCR, entities; Audio and Video — transcribe, summarise transcript.

`omni_browser_ai.js` (`window.OmniBrowserAI`) runs those actions through the Local Engine: `/api/ai/status`, then `ai_generate` (with an image data URL for vision prompts) or `ai_embed`. The engine (`omni_local_ai.py`) picks Ollama (`/api/chat`, default `http://127.0.0.1:11434`) or LM Studio (`/v1/chat/completions`, default `:1234`) per `OMNI_AI_PROVIDER`.

`omni_deepseek_code_ai.js` (`window.OMNI_DEEPSEEK`) adds an "explain code" panel running `onnx-community/deepseek-coder-1.3b-instruct-ONNX` on WebGPU only, with no engine fallback.

Visual and sharing studios

Diagram Forge — `tabDiagramForge`

A browser-only diagram studio with two modes: a visual SVG canvas and Mermaid text diagrams. Code: `diagram_forge_studio.js` (85 KB, 92 functions, `window.REDMARK_DIAGRAM_FORGE`); Mermaid and dagre load on demand through `omni_vendor_loader.js` .

Mode	What you can do	Key functions
Visual canvas (<code>df-mode-canvas</code>)	10 node shapes (rectangle, rounded, ellipse, diamond, parallelogram, database, hexagon, document, sticky note, text), freehand, connectors with curved, elbow or straight routing, images; select, pan, snap and grid; undo/redo, duplicate, group/ungroup, z-order, align and distribute; inspector for fill, stroke, text, size, dash, arrow ends	<code>addNode</code> , <code>addEdge</code> , <code>drawNode</code> , <code>drawEdge</code> , <code>edgeGeometry</code> , <code>smartSnap</code> , <code>alignSel</code> , <code>groupSel</code> , <code>snapshot</code> / <code>undo</code> / <code>redo</code> , <code>updateInspector</code>
Auto-layout	<code>df-autoLayout</code> with direction left→right, top→bottom, right→left, bottom→top	<code>layoutGraph</code> , <code>autoLayout</code> (dagre)
Text diagrams (<code>df-mode-text</code>)	22 Mermaid types (flowchart, sequence, class, state, ER, mind map, architecture, C4, Gantt, timeline, git graph, pie, journey, requirements, Sankey, quadrant, XY chart, packet, block, kanban, event modeling), 5 themes, live render, 9 templates	<code>ensureMermaid</code> , <code>renderMermaid</code> , <code>detectType</code>

Mode	What you can do	Key functions
Convert between modes	Canvas → Mermaid; Mermaid → Canvas for flowchart, state, ER, class and mind map	<code>canvasToMermaid</code> , <code>mermaidToCanvas</code> , <code>parseFlowchart</code> , <code>parseState</code> , <code>parseER</code> , <code>parseClass</code> , <code>parseMindmap</code>
Files	New, Open, Save (native JSON project); export SVG, PNG 2×, transparent PNG, draw.io <code>.drawio</code> , Mermaid <code>.mmd</code> , copy SVG, print/PDF; import <code>.drawio</code>	<code>saveProject</code> , <code>openFile</code> , <code>exportAs</code> , <code>svgToPng</code> , <code>toDrawio</code> , <code>fromDrawio</code> , <code>inflateDrawio</code> , <code>printSVG</code>

File Sharing — `tabFileSharing`

Creates static QR codes and moves files between this computer and a phone over the same Wi-Fi. It is the only studio injected at runtime: `file_sharing_studio.js` (`install`) appends its nav button and panel, so its markup is not in `index.html` (its CSS is).

Updated on 9 Oct 2026 (commit `9d2233e`).

QR Generator

- Content types: link, text, Wi-Fi (`WIFI:` string), contact (vCard 3.0), email (`mailto:`), phone (`tel:`), SMS (`SMSTO:`), location (`geo:`) — built by `qrData()`.
- Design: dot style, corner style, corner dot, error correction L/M/Q/H, foreground, accent (gradient), background, transparent, centre logo.
- Rendering: `renderQr()` uses qr-code-styling 1.9.2 from jsDelivr; offline it falls back to the bundled `qrcode_generator.js` (MIT, Kazuhiko Arase) with a plain style.
`downloadQr` saves PNG or SVG.
- The QR holds the destination itself: no redirect, tracking or expiry.

File Transfer

Direction	Computer side	Phone side
Send to phone	Pick files → each uploads to the engine (<code>/api/upload</code>) → <code>transfer_start</code> with <code>mode: "send"</code> , the file ids and original names	Page lists files with Save buttons; <code>/api/download/<token>/<id></code> streams each with its real name
Receive from phone	<code>transfer_start</code> with <code>mode: "receive"</code> ; received files appear with a Show (in Finder) button	Big picker plus "Photos & videos" and "Take photo"; each file uploads with a progress bar to <code>/api/upload/<token></code> and is saved in <code>~/Downloads/Omni Transfers</code>

- `startTransfer` draws the pairing QR offline (`plainQrSvg`) and shows the PIN, link and a 15-minute countdown.
- `pollTransfer` POSTs `/api/transfer/status` every 1.8 s; `reveal` calls `/api/transfer/reveal` ; `stopTransfer` calls `/api/transfer/stop` .
- Received files are kept after the session ends; temporary copies made for sending are removed.
- The phone page is `TRANSFER_PAGE` inside `omni_local_server.py` (see the Local Engine reference).

System studios

These four studios run and check the engine rather than convert files. Their code is in the inline scripts of `index.html` .

Local Engine — `tabLocal`

One generic form for every native operation.

- `omniLocalOp` picks the job: PDF merge, split, compress, render to PNG ZIP, extract text, extract to DOCX; text/Markdown → DOCX/PDF; scientific inspect and convert; Office convert; EPUB/ebook convert; media convert; image convert/resize; OCR; archive formats.
- Options appear per job: `omniLocalFormat` (one long list of target formats), `omniLocalWidth` and `omniLocalQuality` (images), `omniLocalStart` / `omniLocalEnd` (media trim), `omniLocalDpi` (PDF render), `omniLocalLang` (OCR).

- `omniLocalRun` → for each file `upload` to `/api/upload` → builds the payload (`inputs` for merge and zip, `input` otherwise, plus the options above) → `/api/process` → `downloadFile` from `/api/download/<id>`. `setProgress` shows each stage.
- `omniLocalRefresh` → `updateMode` reads `/api/health`, warns if `engine_api_version` < 4, and fills the header pill (`omniModePill`) with the installed tools.

System Doctor — `tabLocalDoctor`

`doctorBtn` → `doctor` op → `omni_platform.doctor()`, which reports Python packages and native executables separately, so the UI can tell a missing dependency from a real capability.

Diagnostics — `tabDiagnostics`

- `omniRunBrowserTests` → `runBrowserTests`: core UI ready, pdf.js and its worker, JSZip, pdf-lib, Tesseract.js, upload zones, and small conversions in the page.
- `omniRunLocalTests` → `runLocalTests`: connects to the engine, then runs real jobs on generated test files — authenticated upload, ImageMagick conversion, Poppler render and text, qpdf compress/merge/split, Tesseract, LibreOffice CSV → XLSX and CSV → ODS, PDF → PPTX — and skips any whose tool is missing.
- `omniRunAllTests` runs both; `omniExportDiagnostics` saves the log (`runDiagnostics` builds the text report).

Validation Lab — `tabAllTests`

Checks that every control in every studio is wired up.

- `omniRunStructure` → `runStructure`: panels, nav buttons and required elements exist.
- `omniRunOptions` → `runOptions`: walks every `.module-panel` (except the two test studios), exercises each select option and control, and records uncaught errors and rejected promises.
- `runOfficeChecks`: confirms the Office Studio card ids.
- `omniRunAllOptions` → `runAll` runs all three; `omniAllFilter` shows all results or failures only; `omniExportResult` saves them.

Shared shell features

- Command palette (`initCommandPalette`): search box that jumps to any studio.
- Mode pill (`omniModePill`): shows Browser mode, Local Engine connected (with tools), or Local Engine outdated.

- Notifications: `window.omniNotify(message, isError)` from `omni-core.js`, used by every studio.

Local Engine reference

`omni_local_server.py` (1,681 lines, engine build `omni-local-2026.10-complete-language`, API version 4) is a standard-library `ThreadingHTTPServer` that serves the UI and runs 46 named operations through `process_job(payload)`.

Security checks, in order

1. `host_allowed`: the TCP peer must be loopback and the `Host` header must be `127.0.0.1`, `localhost` or `::1` on port 8765; otherwise 403.
2. `origin_allowed`: the `Origin` header must be in `ALLOWED_ORIGINS`. Browsers omit `Origin` on same-origin GETs, so status polling uses POST.
3. `authorize`: `X-Omni-Token` must equal the per-launch `TOKEN` (constant-time compare). The token is injected into `index.html` when the engine serves it and returned by `/api/health` only to an allowed origin.

HTTP routes

Method	Path	Auth	Purpose
GET	<code>/</code> , any static file	host	<code>serve_static</code> : serves repo files, blocks path traversal; rewrites CDN URLs in <code>index.html</code> to <code>vendor/</code> when the 10 offline assets and 12 Tesseract languages are present, and injects <code>window.OMNI_TOKEN</code>
GET	<code>/api/health</code>	origin	<code>process_job({"op": "health"})</code> : engine version, platform, tool versions, Python modules, doctor report, capability lists, token
GET	<code>/api/ai/status</code>	origin	Local LLM availability (Ollama / LM Studio) plus token
GET	<code>/api/download/<id></code>	token	Streams an output file in 1 MB chunks
GET / POST	<code>/api/transfer/status</code>	token	Session state for the File Sharing studio (GET takes <code>?token=</code> , POST a JSON body)

Method	Path	Auth	Purpose
GET	/api/transfer/download/<token>/<id>	token	Streams a file from a transfer session with its original name
POST	/api/upload	token	Raw body + X-Filename ; streams to the temp folder, returns file_id (limit OMNI_MAX_UPLOAD , 8 GB)
POST	/api/process	token	JSON {op, ...} → process_job ; returns {ok, file_id, name, ...} or data
POST	/api/transfer/start / stop / reveal	token	Start or stop a LAN session; show a received file in Finder/Explorer

Files and cleanup. Uploads and outputs live in <system temp>/omni_converter_engine/. register(path) gives each file a random id in the FILES map; get_file(id) resolves it. cleanup_old() runs on every request and deletes files older than 6 hours. Native tools run through run(cmd, timeout) with argument lists, never a shell.

Operations (op values for /api/process)

Group	Ops	Native tool or module	Main params
PDF	pdf_merge , pdf_split , pdf_compress	qpdf	inputs (merge) or input
PDF	pdf_render	Poppler pdftoppm (72-240 DPI)	dpi
PDF	pdf_to_text , pdf_to_docx	Poppler pdftotext	input
PDF	pdf_to_ppt	pdftoppm pages → PPTX	dpi
Office	office_convert	LibreOffice soffice --headless --convert-to	format
Office	text_document	LibreOffice	title , text , format
Ebook	ebook_convert	Calibre ebook-convert	format

Group	Ops	Native tool or module	Main params
Media	<code>media</code>	FFmpeg	<code>format</code> , <code>start</code> , <code>end</code> , <code>scale</code> , <code>audio_only</code> , <code>video_only</code> , <code>keep_video</code>
Image	<code>image</code>	ImageMagick <code>magick</code> / <code>convert</code> , <code>rsvg-convert</code>	<code>format</code> , <code>width</code> , <code>height</code> , <code>quality</code> , <code>density</code> , <code>strip</code>
OCR	<code>ocr</code>	Tesseract	<code>lang</code>
Archive	<code>zip</code> , <code>gzip</code>	Python <code>zipfile</code> (deflate level 9) and <code>gzip</code>	<code>inputs</code>
Archive	<code>archive</code> , <code>archive_extract</code>	7-Zip + <code>tarfile</code>	<code>format</code>
Data	<code>data_convert</code> , <code>data_profile</code> , <code>schema_generate</code>	<code>omni_data_engine</code> (<code>read_data</code> , <code>write_data</code> , <code>profile</code> , <code>schema</code>)	<code>format</code> , <code>source_format</code> , <code>table</code> , <code>dialect</code>
Data	<code>sql_transpile</code>	SQLGlott (<code>transpile_sql</code>)	<code>source</code> , <code>target</code>
Data	<code>nosql_convert</code> , <code>blob_convert</code> , <code>text_lines</code>	<code>omni_data_engine</code>	<code>source</code> , <code>target</code>
Data	<code>scientific_profile</code> , <code>scientific_convert</code>	xarray (NetCDF, GRIB via <code>cfrib</code>), <code>h5py</code> , <code>Astropy</code> <code>FITS</code> , <code>cdflib</code>	<code>source_format</code> , <code>format</code> , <code>max_rows</code>
Platform	<code>doctor</code> , <code>capability</code> , <code>privacy_scan</code> , <code>data_clean</code> , <code>batch_convert</code> , <code>database_tables</code> , <code>database_query</code>	<code>omni_platform</code>	<code>actions</code> , <code>operation</code> , <code>job_op</code> , <code>inputs</code> , <code>params</code>

Group	Ops	Native tool or module	Main params
Code	<code>code_capabilities</code> , <code>code_execute</code>	<code>omni_code_runner</code>	<code>language</code> , <code>source</code> , <code>stdin</code> , <code>timeout</code>
Language	<code>language_capability</code> , <code>language_translate</code> , <code>language_transliterate</code>	<code>omni_language_engine</code>	<code>source</code> , <code>target</code> , <code>text</code> , <code>max_new_tokens</code> , <code>topk</code>
AI	<code>ai_status</code> , <code>ai_chat</code> , <code>ai_generate</code> , <code>ai_embed</code>	Ollama or LM Studio	<code>prompt</code> , <code>messages</code> , <code>model</code> , <code>max_tokens</code> , <code>temperature</code> , <code>image_data_url</code> , <code>text</code>
Social	<code>social_download</code>	yt-dlp (Python module)	<code>url</code> , <code>mode</code> , <code>quality</code> , <code>info_only</code>
Transfer	<code>transfer_start</code> , <code>transfer_status</code> , <code>transfer_stop</code>	built-in	<code>mode</code> , <code>file_ids</code> , <code>names</code> , <code>ttn</code> , <code>token</code>
Engine	<code>health</code>	—	—

LAN transfer server

`start_transfer_server()` opens a second `ThreadingHTTPServer` on `0.0.0.0`, port `8766-8777`, only while a session exists; `stop_transfer_server_if_idle()` shuts it down when the last session ends or expires. Sessions live in `TRANSFER_SESSIONS` keyed by a random URL-safe token of about 32 characters, with a 6-digit PIN and a TTL of 60-3,600 s (default 900).

Route on the LAN server	Purpose
<code>GET /share/<token></code>	The phone page (<code>TRANSFER_PAGE</code>): mobile-first, dark-mode aware, PIN and countdown, file picker built once and never re-rendered

Route on the LAN server	Purpose
<code>GET /api/status/<token></code>	Mode, PIN, expiry and file list; marks the phone as connected
<code>GET /api/download/<token>/<id></code>	Streams a shared file (<code>stream_file</code>) with <code>filename*=UTF-8''</code> for non-ASCII names
<code>POST /api/upload/<token></code>	Receive mode only: streams the body to <code><name>.part</code> , then renames into <code>transfer_inbox()</code> (<code>~/Downloads/Omni Transfers</code>) with a unique name (<code>transfer_unique_path</code>)

`lan_ip()` finds the computer's Wi-Fi address; `transfer_start` warns when it can only find `127.x`.

Browser modules reference

33 JavaScript files and 5 stylesheets make up the browser side. Every module is a self-running IIFE; modules share work only through the `window` globals listed below and `window.omniNotify`.

File	Size	Global	Role
<code>omni-core.js</code>	105 KB	<code>__OMNI_CORE_READY</code>	Original studio wiring: PDF Suite, compression, OCR, audio/video basics, EPUB, drag-and-drop (<code>setupDragDrop</code>), <code>omniNotify</code> , <code>omniDownload</code> , <code>formatFileSize</code>
<code>omni_browser_docs.js</code>	63 KB	<code>OMNI_DOCS</code>	Document model: DOCX/ODT/RTF/HTML/MD/TXT/PPTX readers; PDF/DOCX/ODT/HTML/MD/TXT writers; sheet conversion
<code>omni_browser_epub.js</code>	26 KB	<code>OMNI_EPUB</code>	EPUB, FB2, CBZ read; EPUB 3, FB2, RTF, HTMLZ write
<code>office_tools_studio.js</code>	123 KB	<code>REDMARK_OFFICE_TOOLS</code>	Word/Excel/PowerPoint/Notepad editors
<code>office_find_replace.js</code>	25 KB	<code>OMNI_FIND_REPLACE</code>	Find & Replace inside Office XML packages
<code>omni_browser_images.js</code>	26 KB	<code>OMNI_IMAGES</code>	Image decode and encode (AVIF, GIF, TIFF, BMP, ICO, TGA, PDF, SVG), DPI, size targets
<code>photo_editor.js</code>	39 KB	<code>OMNI_PHOTO_EDITOR</code>	Single-photo editor
<code>omni_browser_ocr.js</code>	8 KB	—	Multi-image and PDF OCR, clean-up, searchable PDF

File	Size	Global	Role
omni_browser_audio.js	25 KB	OMNI_AUDIO	Waveform, WSOLA time-stretch, WAV/MP3/M4A/Opus/AIFF/AU export
omni_browser_video.js	24 KB	OMNI_VIDEO	Mediabunny cut/convert, frames, GIF
omni_browser_archive.js	20 KB	OMNI_ARCHIVE	7z-wasm worker: 7z, AES ZIP, TAR variants, RAR/ISO/CAB extract
omni_browser_studio_plus.js	36 KB	OMNI_BROWSER_PLUS	Data Clean, privacy scan, Batch Lab, SQLite/Parquet, text/BLOB bridges, rule-based transliteration
universal_data_studio.js	23 KB	OMNI_UNIVERSAL_DATA	Universal Data parsing, charts, maps, sonification, export
omni_universal_data_formats.js	26 KB	OMNI_UDS_FORMATS	Extra formats and engine hand-off for Universal Data
omni_sleep_hypnogram.js	18 KB	OMNI_SLEEP_HYPNOGRAM	EDF/EDF+ sleep stage decoding and metrics
bi_code_studio.js	30 KB	OMNI_BI_INTERCHANGE , OMNI_CODE_STUDIO	BI Interchange and Code Studio translator
diagram_forge_studio.js	83 KB	REDMARK_DIAGRAM_FORGE	Diagram canvas, Mermaid, draw.io
social_media_studio.js	22 KB	OMNI_SOCIAL_MEDIA_DOWNLOADER	Link analysis and media download
social_resolver_config.js	<1 KB	OMNI_SOCIAL_RESOLVER_URL	URL of the optional resolver service (empty by default)
file_sharing_studio.js	27 KB	—	File Sharing studio (QR + LAN transfer)
qrcode_generator.js	55 KB	qrcode	Bundled MIT QR encoder (offline pairing QR)
rag_studio.js	31 KB	__omniTransformers	Private RAG
rag_embed_worker.js	1 KB	—	Embedding Web Worker
omni_assistant.js	25 KB	—	Site-wide assistant
omni_local_ai.js	3 KB	OmniLocalAI	Transformers.js embeddings + chat runtime
omni_browser_ai.js	4 KB	OmniBrowserAI	AI Assist calls to the engine (Ollama / LM Studio)
omni_browser_ai_ui.js	20 KB	—	AI Assist buttons in each studio
omni_deepseek_code_ai.js	5 KB	OMNI_DEEPSEEK	DeepSeek Coder 1.3B on WebGPU
omni_language_engine.js	5 KB	OmniLanguageEngine	Translation / transliteration client
omni_language_browser.js	3 KB	OMNI_LANGUAGE_BROWSER_ADAPTER	Opus-MT Hindi ↔ English in the browser
omni_language_studio.js	4 KB	—	Language Engine panel wiring
omni_vendor_loader.js	4 KB	OMNI_VENDOR	On-demand library loader, vendor / first then pinned CDN

File	Size	Global	Role
<code>omni_render_cloud.js</code>	8 KB	<code>OmniRenderCloud</code>	Opt-in Cloud button; sends jobs to <code>OMNI_CLOUD_ENGINE_URL</code> (default <code>https://omni-cloud-engine.onrender.com</code>) with <code>X-Omni-Cloud-Token</code> kept in <code>sessionStorage</code>

Libraries loaded by `omni_vendor_loader.js` : mammoth 1.13.0, docx 9.9.0, PptxGenJS 4.0.1, ExcelJS 4.4.0, fast-formula-parser 1.0.19, Formula.js 4.6.1, DOMPurify 3.4.16, Mermaid, dagre, SheetJS, JSZip. Other engines load their own codecs from jsDelivr on first use: heic2any, UTIF, pako, gifenc, @jsquash/avif, lamejs, mp4-muxer, webm-muxer, Mediabunny, 7z-wasm, hyparquet, sql.js, Transformers.js 4.3.0.

Stylesheets: `omni-ui-revamp-color.css` (47 KB, the suite-wide theme and per-studio colours), `office_tools_studio.css`, `diagram_forge_studio.css`, `photo_editor.css`, `office_find_replace.css`; the rest of the CSS, including File Sharing's, is inline in `index.html`.

Python engines

Six Python modules sit behind the Local Engine (about 4,600 lines of Python in the repo in total). Each can also be imported on its own; `omni_sdk` re-exports the data and platform functions.

`omni_data_engine.py` (423 lines)

Structured, semi-structured and scientific data.

- `read_data(path, fmt, table)` → pandas DataFrame from CSV, TSV, TXT, JSON, JSONL, YAML, XML, HTML, XLSX, XLS, XLSB, ODS, Parquet, HDF5, Feather/Arrow, ORC, Avro, SQLite, DuckDB, SQL and MessagePack (`DATA_FORMATS` lists 32 extensions including Stata, SAS and SPSS).
- `write_data(df, out, fmt, table, dialect)` writes the same family; `dataframe_sql` emits `CREATE TABLE + INSERT` with `_ident`, `_type`, `_literal` per dialect.
- `profile`, `schema`, `transpile_sql` (SQLGlot across 24 `SQL_DIALECTS`, from Oracle and T-SQL to ClickHouse and Tableau), `nosql_convert` (with `_ddb_wrap` / `_ddb_unwrap` for DynamoDB), `blob_convert`, `text_lines`.

- `scientific_profile / scientific_convert` : NetCDF and GRIB via xarray (cfgrib), HDF5 via h5py, FITS via Astropy, CDF via cdflib; capped by `max_rows` (default 2,000,000).
- `module_status()` reports which optional Python packages are importable; `_mod(name)` imports lazily.

omni_platform.py (159 lines)

- `doctor()` — Python packages and native executables with versions (`_version`).
- `capability(source, target)` — returns a route for a source/target pair: browser, browser-first (engine as fallback for scale) or local, with a reason.
- `privacy_scan(path, max_bytes=8 MB)` and `clean_dataframe(df, actions)` — back Data Clean.
- `database_query(url, query, params) / database_tables(url)` — read-only SQL. `_validate_database_url` allows 14 schemes (SQLite, PostgreSQL, MySQL/MariaDB, MSSQL, Oracle, Snowflake, BigQuery, Databricks, DuckDB, Trino, ClickHouse, Redshift), enforces `OMNI_ALLOWED_DB_HOSTS`, can disable localhost with `OMNI_ALLOW_LOCAL_DB=0`, and refuses hosts that resolve to private, loopback or link-local addresses.
- `batch_manifest(results)` — the JSON report Batch Lab downloads.

omni_code_runner.py (240 lines)

- `available_languages()` finds toolchains on PATH for 13 languages: Python, JavaScript, TypeScript, Bash, Ruby, PHP, C, C++, Java, Go, Rust, Swift, Kotlin.
- `execute(language, source, stdin, timeout)` writes the source into a fresh temp folder, compiles if needed and runs it with a timeout of 1–20 s (default 8). `_env` points HOME and TMP at that folder and strips proxy variables. This is isolation, not a sandbox: code runs as your user.

omni_language_engine.py (143 lines)

- `translate(text, source, target) → _route` picks en→indic, indic→en or indic→indic → `_load_translation` loads the IndicTrans2 model from `language-models/` → `_translate_indictrans` with IndicTransToolkit pre/post-processing.
- `transliterate(text, source, target="Latn", topk=4) → IndicXlit` in either direction.

- `capability()` / `config()` report which models are installed. Paths can be overridden with `OMNI_IT2_EN_INDIC_MODEL` , `OMNI_IT2_INDIC_EN_MODEL` , `OMNI_IT2_INDIC_INDIC_MODEL` .

omni_local_ai.py (146 lines)

- `status()` lists models on Ollama (`/api/tags`) and LM Studio (`/v1/models`).
- `_provider()` resolves `OMNI_AI_PROVIDER=auto` to whichever is running.
- `chat` , `generate` (text or vision with images) and `embed` (needs `OMNI_AI_EMBED_MODEL` , for example `nomic-embed-text`).

omni_sdk package and CLI

Published as `omni-converter-studio 0.1.0` (`pyproject.toml` , author Raghav Raman). `omni_sdk/__init__.py` re-exports `read_data` , `profile` , `schema` , `transpile_sql` , `scientific_profile` , `scientific_convert` , `doctor` , `capability` , `clean_dataframe` , `privacy_scan` , `database_query` , `database_tables` , `batch_manifest` , `SQL_DIALECTS` and `DATA_FORMATS` .

```
omni-engine doctor      # inspect installed Python and native capabilities
omni-engine version     # SDK version
omni-engine serve       # start the Local Engine
```

```
from omni_sdk import profile, transpile_sql
profile("sales.parquet")
transpile_sql("SELECT TOP 5 * FROM t", "tsql", "postgres")
```

Deployment and packaging

The suite ships five ways: the GitHub Pages site, a Local Engine container on GHCR, an optional Render cloud engine, an optional Render social resolver, and two packages (Python SDK, GitHub npm package).

Target	Built from	Notes
GitHub Pages site	<code>.github/workflows/pages.yml</code> on push to the main branch	Publishes the repo as a static site at <code>https://theraghavraman.github.io/omni_suite_light/</code> ; browser mode only

Target	Built from	Notes
Local Engine container	Root <code>Dockerfile</code> (Python 3.13 slim) via <code>publish-container.yml</code> to <code>ghcr.io</code>	Installs FFmpeg, qpdf, Poppler, ImageMagick, libsvg, Ghostscript, Tesseract (eng, hin), LibreOffice, Calibre, Pandoc, 7-Zip, plus HDF5/NetCDF/eccodes libraries and <code>requirements-local.txt</code> ; binds <code>127.0.0.1:8765</code> ; health check on <code>/api/health</code>
Render cloud engine	<code>omni-cloud-engine/Dockerfile</code> + <code>app.py</code> (Flask, 298 lines) + <code>render.yaml</code>	Service <code>omni-cloud-engine</code> , free plan; <code>OMNI_CLOUD_TOKEN</code> required, <code>CLOUD_ALLOWED_ORIGINS</code> = the Pages site, <code>MAX_UPLOAD_MB</code> 500, <code>MAX_JOBS</code> 25, <code>RATE_LIMIT_PER_MINUTE</code> 12
Render social resolver	<code>social-resolver/</code> (FastAPI, 913 lines) + <code>render.yaml</code>	Service <code>omni-social-resolver</code> ; GET <code>/api/health</code> , POST <code>/api/resolve</code> , GET <code>/api/download</code> ; public URLs only, 15-minute and 300 MB limits; paste its URL into <code>social_resolver_config.js</code>
Python SDK	<code>pyproject.toml</code> → <code>omni-converter-studio</code> 0.1.0	Modules <code>omni_data_engine</code> , <code>omni_platform</code> , <code>omni_local_server</code> + package <code>omni_sdk</code> ; CLI <code>omni-engine</code> ; see <code>README.pypi.md</code>
npm package	<code>package.json</code> → <code>@theraghavraman/omni-universal-data-studio</code> 1.0.0 via <code>publish-npm.yml</code> (manual, version input)	Ships only <code>universal_data_studio.js</code> ; published to GitHub Packages

Cloud engine operations (`POST /api/process` with `operation`, `options` and files; progress at `GET /api/jobs/<id>`): `ffmpeg` / `media-transcode`, `office-convert`, `pdf-batch`, `image-batch`, `ocr`, `archive`, `metadata`, `data-profile`, `data-clean`, `sql-simulate`, `code-format`, `code-clean`, `rag-ingest`, `vector-search`, `batch-pipeline`, `scheduled-pipeline`, `social-batch`, `diagnostics`. The browser bridge is `omni_render_cloud.js`; the token stays in `sessionStorage`.

CI (`local-engine-check.yml`, on every push and pull request) runs on Ubuntu, Windows and macOS:

1. Compile-check every Python engine and `node --check` every studio script (including `file_sharing_studio.js` and `qrcode_generator.js`).
2. `.github/scripts/studio_wiring_check.mjs` — Office Tools and Diagram Forge wiring.
3. `.github/scripts/assistant_rag_regression.mjs` — assistant retrieval regression.

4. Install native tools per OS, verify executables, install `requirements-local.txt`.
5. `.github/scripts/native_smoke_test.py` — representative native conversions.
6. Dependency audit. `.github/scripts/omni_contract_audit.py` is the static contract audit mentioned in the README.

Requirement files: `requirements-local.txt` (complete profile used by the installers), `requirements-data.txt`, `requirements-extended.txt` (database drivers, Polars, SciPy, SymPy and more), `requirements-language.txt` (PyTorch, Transformers, IndicTransToolkit), `requirements-vision.txt` (EasyOCR).

File index

Every file in the repository root and its folders, apart from the JavaScript and CSS files already listed under Browser modules reference. Sizes are on-disk sizes rounded up to 4 KB blocks.

File or folder	Size	Role
<code>index.html</code>	568 KB	The whole app: 25 studio panels, nav, inline CSS, 24 inline scripts
<code>omni_local_server.py</code>	96 KB	Local Engine HTTP server, 46 ops, LAN transfer server and phone page
<code>omni_data_engine.py</code>	24 KB	Data and scientific conversion engine
<code>omni_platform.py</code>	12 KB	Doctor, capability routing, privacy, cleaning, read-only SQL, batch manifest
<code>omni_code_runner.py</code>	12 KB	Local code execution for 13 languages
<code>omni_language_engine.py</code>	8 KB	IndicTrans2 and IndicXlit
<code>omni_local_ai.py</code>	8 KB	Ollama / LM Studio client
<code>omni_bootstrap.py</code>	8 KB	One-command launcher: verify, install, replace stale engine, start
<code>prepare_offline.py</code>	8 KB	Downloads pinned browser assets into <code>vendor/</code>
<code>prepare_language_models.py</code>	4 KB	Fetches language models into <code>language-models/</code>

File or folder	Size	Role
<code>verify_local_environment.py</code>	8 KB	Strict post-install check
<code>Omni.command</code> , <code>Omni.bat</code>	4 KB each	Double-click entry points
<code>start_omni.command</code> , <code>start_omni.bat</code>	4 KB each	Pick Python, run the bootstrap
<code>setup_and_start.command</code> , <code>setup_and_start.bat</code>	4 KB each	Installer then launcher
<code>install_system_tools.command</code>	8 KB	macOS/Linux native tool + venv installer
<code>install_windows.ps1</code>	8 KB	Windows installer through winget
<code>Dockerfile</code>	4 KB	Local Engine container image
<code>pyproject.toml</code>	4 KB	Python package <code>omni-converter-studio</code>
<code>package.json</code>	4 KB	npm package for Universal Data Studio
<code>requirements-*.txt</code> (5 files)	4 KB each	Python dependency profiles: local (44 lines), data, extended, language, vision
<code>README.md</code>	32 KB	User guide: modes, setup, offline, troubleshooting
<code>LOCAL_ENGINE.md</code>	12 KB	Full local/offline edition details
<code>README.npm.md</code> , <code>README.pypi.md</code>	4 KB each	Package READMEs
<code>THIRD_PARTY_NOTICES.md</code>	8 KB	Licences for every bundled or loaded library and model
<code>LICENSE</code>	4 KB	MIT
<code>favicon.svg</code> , <code>favicon.ico</code> , <code>favicon-32.png</code> , <code>apple-touch-icon.png</code>	4-16 KB	Icons
<code>.dockerignore</code> , <code>.gitignore</code>	<1 KB	Excludes <code>vendor/</code> , <code>.venv/</code> , caches and temp folders from images and git

File or folder	Size	Role
<code>knowledge-base/</code>	—	Assistant knowledge: <code>omni-suite.md</code> , <code>omniconverter-guide.md</code> , <code>manifest.json</code> (version 3), and a sample Project Gutenberg PDF
<code>language-models/</code>	—	Model folder for the Language Engine (only a README is committed)
<code>omni_sdk/</code>	—	<code>__init__.py</code> , <code>cli.py</code> , <code>py.typed</code>
<code>omni-cloud-engine/</code>	—	Render cloud engine: <code>app.py</code> , <code>Dockerfile</code> , <code>render.yaml</code> , <code>requirements.txt</code>
<code>social-resolver/</code>	—	Render social resolver: <code>app.py</code> , <code>Dockerfile</code> , <code>render.yaml</code> , <code>requirements.txt</code> , <code>runtime.txt</code> , <code>README.md</code>
<code>.github/workflows/</code>	—	<code>local-engine-check.yml</code> , <code>pages.yml</code> , <code>publish-container.yml</code> , <code>publish-npm.yml</code>
<code>.github/scripts/</code>	—	<code>studio_wiring_check.mjs</code> , <code>assistant_rag_regression.mjs</code> , <code>native_smoke_test.py</code> , <code>omni_contract_audit.py</code>
<code>vendor/</code> (generated, git-ignored)	—	Offline copies of pdf.js, JSZip, pdf-lib, Tesseract + languages, SheetJS, js-yaml and on-demand libraries
<code>.venv/</code> (generated, git-ignored)	—	Python environment created by the installers